

VNSecure: An explainable virtual network attack detection framework at VMM-Layer in virtualization environment

Aditya Nautiyal
Doon University, Dehradun
India
adityanautiyal5@gmail.com

Shubhangi Saklani
Doon University, Dehradun
India
shubhusaklani11@gmail.com

Aditi Pant
Doon University, Dehradun
India
aditipant2223@gmail.com

Yutika Agarwal
NorthCap University, Gurgaon
India
yuti.adm@gmail.com

Avantika Gaur
Doon University, Dehradun
India
avigaur99@gmail.com

Preeti Mishra*
Doon University, Dehradun
India
dr.preetimishranit@gmail.com

ABSTRACT

In recent years, virtualization has become popular with the usage of emerging technologies in different domains. With the increasing number of attacking incidents in recent times, virtualization security has become one of the primary focus of research. The traditional network attack detection systems are inefficient enough to detect attacks over the virtual network. In this paper, we have proposed a virtual network security framework, called VNSecure that detects malicious network activities by analyzing virtual machine (VM) traffic profile. VNSecure operates on a hypervisor layer and has access to both underlying hardware and guest operating system. It does VM-level activity analysis from the privileged domain of the hypervisor, serving as the main line of defense against intrusions at the virtual network level. Initially, VM traffic validation is performed to detect spoofing attacks by analyzing the traffic captured at the backend driver of the virtual network interface of the monitored VM. To perform detailed behavior analysis, a deep learning approach is used to learn and detect VM-specific network attacks. On detection of malicious traffic, an alert is raised to the administrator. VNSecure then carries out essential mitigation to lower the risk and store the occurrence of malicious packets in its database. One public dataset and one self-generated attack dataset have been used to validate all the modules of the proposed framework and the results seem to be promising.

ACM Reference Format:

Aditya Nautiyal, Shubhangi Saklani, Aditi Pant, Yutika Agarwal, Avantika Gaur, and Preeti Mishra. 2023. VNSecure: An explainable virtual network attack detection framework at VMM-Layer in virtualization environment. In *2023 Fifteenth International Conference on Contemporary Computing (IC3-2023) (IC3 2023)*, August 03–05, 2023, Noida, India. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3607947.3607974>

*Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

IC3 2023, August 03–05, 2023, Noida, India

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0022-4/23/08...\$15.00

<https://doi.org/10.1145/3607947.3607974>

1 INTRODUCTION

Virtualization has become one of the most popular technologies nowadays. Most of companies are shifting various services and data to virtual infrastructure. However, the advancement in technology and the widespread use of network have caused a great threat to confidential information and personal data. The economic times reported one recent attacking incident that has bypassed the network security defenses of AIIMS Delhi and compromised the data of over 40 million records¹. The government of Belgium has suffered cyber attacks thrice by Chinese hackers². Uber's virtual private network has also been infected by various cyber-attacks recently³. These attacking incidents have raised serious security questions on the safety of sensitive information of virtual/physical infrastructure. Especially there is a need to develop more specialized security solutions for the virtualization-based ecosystem.

The identification of malicious activities within a network can be effectively achieved through the implementation of an intrusion detection system (IDS), which is widely recognized as a popular defensive solution [11]. It is essential to monitor the network and identify suspicious activities for preventing the malicious traffic that may cause harm to the network/data. IDS can be classified into the following types: host-based IDS (HIDS) and network-based IDS (NIDS). A HIDS can be deployed in a VM/physical host where it monitors and detects intrusions by analyzing the audit logs and other system activities. On the contrary, NIDS monitors the network traffic for malicious activities by analyzing the network activities of virtual/physical hosts or network points.

The IDS systems make use of two popular detection mechanisms: misuse detection and anomaly detection. Misuse detection is capable of detecting known attacks and their variants. A supervisor ML approach also falls under this category. Anomaly detection involves the detection of unknown attacks. The unsupervised/semi-supervised ML models fit under this category. Network introspection-based IDSes analyze the network activity logs collected at the hypervisor (outside the monitored VM) by sniffing the virtual network interfaces and have been least explored.

¹<https://ciso.economictimes.indiatimes.com/news/aiims-ransomware-attack-what-it-means-for-health-data-privacy/96538957>

²<https://newsroom.mastercard.com/news/europe/en/newsroom/press-releases/en/2022/january/mastercard-reveals-record-levels-of-cybercrime-in-belgium-during-the-pandemic/>

³<https://www.cshub.com/attacks/news/iotw-over-77000-uber-employee-details-leaked-in-data-breach>

Various approaches have been proposed in the area of NIDS to analyze the network activity. For example, Han and Pak[5] proposed a NIDS which minimizes memory consumption for feature generation and achieves very high detection performance without detection delay. The authors assumed that session packets are received without loss. In a real-life scenario, the NIDS may fail as data packets may be lost. Yu et al. [18] proposed flexible and robust NIDS based on Recurrent Neural Network (RNN) with a multi-classifier to generate a real-time detection model and achieved 98% accuracy. Kim and Pak[7] proposed ML-based NIDS that identifies intrusions early in the active session. The proposed technique is highly dependent on the dataset given to it and is based on an early-stage detection model. But, in real-world scenarios, intrusion activities can bypass NIDS by revealing their actual nature after some time. Mishra et al.[12] a security strategy called Malicious Network Packet Detection (MNPd) has been presented to oversee virtual machines (VMs) externally to the network. The approach involves analyzing network traffic behavior at CNS and offers primary defense. Zhao et al.[19] proposed a NIDS based on a virtual computing environment(VNIDS). The proposed framework consists of a cross-domain communication module for the transmission of information between the VM and VNIDS and splits the Xen VMM as client and server, and builds a connection between them. If the client VM accesses the machine memory page that does not belong to it, project faults will arise, and the approach might prove inefficient.

The existing works in the virtualization domain [12] [14] have mostly used the traditional/conventional IDS mechanism, which may not capture the complex behavior of evolving attacks. Moreover, deployment of conventional IDSes on every VM [6] of a virtualized server makes them a less cost-effective solution and also diminishes their robustness. The performance of existing hypervisor-based approaches [4][13] which have been validated with older datasets with recent feature statistics of evolving attack patterns, is difficult to assess. The statistical properties of the new evolving attacks and respective attacking strategies are different, making the older datasets less suitable for analysis. In addition, the ML models have been used as black box models in the existing hypervisor-based IDS [12], making their predictions less interpretable.

This paper proposes an explainable network introspection-based attack detection model called *VNSecure* to detect attacks in the virtualization environment. Initially, the malware executables are injected and executed directly from the privileged domain into the VM using the introspection mechanism [8]. To monitor the network activity of a VM from the hypervisor, the virtual network interface is sniffed, capturing the network traffic which is then pre-processed and sent to the network traffic validator. The validator examines the traffic to detect any IP/MAC address spoofing from untrusted domains. In addition, a network behavior analyzer is activated to classify the network traffic flow using a deep learning algorithm (deep neural network) to identify any malicious behavior in the network packets. On detection of suspicious network traffic flow, an alert is raised to the administrator. Two datasets have been considered for validating the approach. Initially, CICIDS2017 [15] is used (provided by the Canadian Institute of cyber security) for analysis. Furthermore, a self-generated network attack dataset is created from a malware executable (provided by the San Jose

State University, California [17]) in the test bed setup of our lab. The results are interpreted using the explainable AI (XAI) libraries (SHAP [9] and Eli5⁴).

The important contributions of the paper are as follows:

- To design and implement a robust and explainable security model for network attack detection at hypervisor layer.
- To design and implement a spoofing detection method at VMM for VM traffic validation that uses network introspection to capture VM network artifacts from privileged domain of the hypervisor.
- To conduct behaviour analysis of virtualized network traffic using deep neural network and explain the results using ELI5 and SHAP libraries.
- To validate the proposed framework using amazon web services (AWS) based CICIDS2017 dataset and self-generated network traffic dataset and provide comparative discussion with state-of-the-art approaches.

The rest of the paper is organized as follows: Section 2 briefly describes the background of the proposed research under virtualization technology. Section 3 describes the proposed model and all its modules. Section 4 provides the case study in which we have discussed the experiment and results. The Section ?? provides the acknowledgement. Finally, we have concluded our paper in Section 5.

2 BACKGROUND: XEN NETWORKING

This section explains various important information related, including the Xen API, databases, network allowing drivers, and DHCP servers which are useful for network introspection. Xen is a bare-metal hypervisor (type I) that is very popular in most of cloud computing service provisioning frameworks. The Xen hypervisor's trusted, authorised domain is Dom0 that boots up initially and manages the administrative activities of the virtualized server. DomU is an untrusted user domain where user applications are deployed and provisioned. The end users may have access to the DomU domain but can not access the Dom0 domain for security reasons. The administrator of the virtualized server may have access to a privileged domain.

The bare-metal machine consists of two different APIs: (i) Xen-API/Xen management API and (ii) Xen system-call/hyper-call API. Xen utility tools use the former, and guest machines use the latter [3]. The Xen-API serves as a connection between low-level daemons and user-space applications. When a user space application issues a request, it is processed and transmitted to the Dom0 kernel through the management functions, and a request for service/resource is provisioned to VM. The real/native network interface device drivers are installed at Dom0. Xen configures the NIC driver by parting it as the front-end driver, which is installed on the DomU, and as the back-end driver, which is installed on Dom0.

Xen offers an ethernet bridge interface called *virbr0* that connects the virtual NICs with the physical NIC. This bridge works by multi-plexing and demultiplexing traffic between the physical NIC and virtual NICs and vice versa. When a user virtual machine wants to forward data to other machines, the traffic is forwarded by the FrontendDriver to the BackendDriver (at Dom0). All such

⁴<https://eli5.readthedocs.io/en/latest/overview.html>

VM's requests are lined-up in the driver domain of Dom0 and subsequently forwarded to the actual physical device's driver. The virtual network traffic of the monitored VM can be captured at Dom0 by sniffing the respective BackendDriver (of the monitored machine). Xen uses the naming convention for the backend driver as *vif.DomID.DeviceID*, where DomID refers to the domain-id of the monitored VM.

Virtual NICs that are connected to physical NICs using the ethernet bridge interface provided by Xen are known as *virbr0*. The physical and virtual NIC traffic is multiplexed and de-multiplexed for *virbr0*. Traffic is routed from the FrontendDriver to the BackendDriver (at Dom0) when a user VM needs to send information to other computers. All of these VM requests are queued in Dom0's driver domain before being sent on to the real physical device driver. The observed VM's virtual network traffic can be recorded at Dom0 by sniffing the relevant BackendDriver (of the monitored machine).

The Xen server utilizes the *dnsmasq* service located in the Dom0 of virtual servers to assign private IP addresses to each DomU VM running on top of hypervisor⁵. As such, in our implementation, we have also adopted the *dnsmasq* service to configure a Dynamic Host Configuration Protocol (DHCP) server located in the Dom0 of the Hypervisor. This DHCP server is capable of providing support for various functions, including Domain Name Server (DNS), and network boot. Moreover, the *Dnsmasq* DHCP server is capable of supporting both static and dynamic leases, multiple networks, and IP address ranges.

3 VNSECURE: PROPOSED DESIGN FRAMEWORK

This section explains the design modules of the proposed security framework *VNSecure* in detail. The *VNSecure* is composed of four different detection modules, namely, virtual network traffic capturing (VNTC), virtual network traffic validator (VNTV), network traffic behavior analyzer (NTBA), and alert, report, and mitigation (ARM) module. The first module VNTC is used to capture the virtual network's traffic in the form of packets (*.pcap* logs). VNTV module is designed to do primary scanning by validating the network traffic for the presence of any spoofed MAC/IP in traffic flow. NTBA functions in two stages: pre-processing and analyzing behavior for detecting suspicious activities. ARM module takes the necessary action to warn the administration about the compromised VM and block it. The deployment of the modules is shown in Figure 1. Let us now discuss various modules in detail:

3.1 Virtual network traffic capturing (VNTC)

VNTC is one of the most crucial modules of the proposed framework, deployed at the privilege domain (Dom0) of the hypervisor. VNTC sniffs the network packets flowing through the virtual bridge and passing via a virtual network interface (*vif.DomID, DevID*). The virtual networking concept is explained in detail in Section 3. The *VNSecure* utilizes *tcpdump*⁶, an open-source network sniffing tool for this purpose. For each of the monitored VMs, packet sniffing is carried out concurrently by listening to the machine's relevant Virtual Network Interface (VNI); with that, their VM profiles are

built. Each profile represents the network behavior statistics that are stored in log files with the extension as *.pcap*. The captured data represents the information in the form of traffic flows (having packet header and payload values).

3.2 Virtual network traffic validator (VNTV)

Security measures applied at the VM level can be bypassed if the malicious network flow is foiled up as legitimate. Therefore, it is impossible to completely trust VM-level security methods that guarantee that the user is only producing legitimate packets. There might be a possibility that a compromised VM may start flooding the virtual network bridge with forged source addresses, triggering a denial-of-service attack. It is difficult to identify such attacks when co-located VMs are communicating since the traffic will never pass through the network server; rather, it will be passed via network event channels of the hypervisor. In the proposed framework, a security measure is proposed at the hypervisor layer to validate the virtual network traffic in such cases. VNTV module of the proposed framework is responsible for validating the network traffic passing via the back-end driver of virtual network interface (VNI) (*vifA.B*) attached to the bridge interface (*virbr0*).

It is highlighted that VM's network traffic flows through the virtual interface prior to reaching the physical interface. The traffic's metadata of the real VM (IP /MAC) gets lost when it reaches to the physical interface. The traffic captured by sniffing the physical interface by traditional IDS won't be able to provide the VM-specific information, and it fails to do fine-granular analysis. The proposed *VNSecure* handles this issue by working at the virtual network bridge level. As mentioned in Section 2, each VNI (*vif.DomID.DevID*) is linked to a VM (having some DomID). Hence, it is earlier to trace the VM whose traffic is passing through the specific VNI by looking at the VNI name. Once the domain ID (named as DomID) is extracted, Xen API commands are used to map the domain ID to the domain name to determine which domain (such as Windows7-VM1 or Ubuntu-VM2) is generating the traffic. The traffic collected at VNI is useful to extract the network information, getting generated at VM-level such as IP, MAC, etc. There are chances that the attacker spoofs VM-level information (specifically IP/MAC).

Hence, the DHCP service (*dnsmasq*) in the Xen server provides an original IP and MAC for each VM. Dom0 hosts the *Dnsmasq* service, which is set up for each VM network. The configuration file is located at */etc/dnsmasq.conf*. The MAC and IP addresses of the VM can be determined using this mapping. Moreover, IP addresses are assigned to VMs by DHCP. The MAC address is provided to a VM which is mentioned in */etc/xen/Ubuntu-Guest-VM1.cfg*. The configuration data related to the MAC address is maintained in the Xenstore [10]. The DHCP service stores the networking configurations in (*/var/lib/libvirt/dnsmasq/virbr0.status*) stored during each VM booting procedure. The entries of MAC and IP addresses are introspected by *VNSecure*, which is maintained by one of the services of DHCP to extract the configuration information.

The IP and MAC of the receiving packets from DomU are fetched by the VNTC module and are compared with the stored IP /MAC from the IP MACTable. The virtual machine is marked as a compromised VM if the fetched IP /mac does not match the registered IP /MAC. And the status is passed to the "alert and mitigation module."

⁵<https://thekelleys.org.uk/dnsmasq/docs/dnsmasq-man.html>

⁶<https://www.tcpdump.org/>

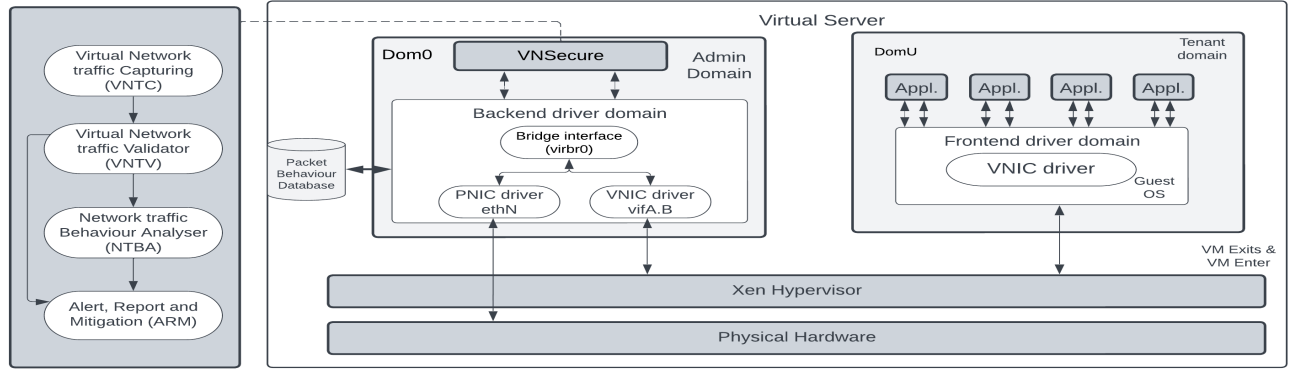


Figure 1: Proposed Architecture: VNSecure

Algorithm 1 Virtual Network Traffic Validator Algorithm

Require: $IPMACTable \leftarrow \text{var/lib/libvirt/dnsmasq/virbr0.status}$

```

1: procedure VNTV()
2:   procedure CHECKMAC()
3:      $R\_MAC \leftarrow \text{ExtractMAC}(IPMACTable)$ ;
4:     if  $R\_MAC == mac.vif\_id$  then ▷ check real mac equals to mac in packet
5:        $flag\_id \leftarrow 0$ 
6:     else
7:        $flag\_id \leftarrow 1$ 
8:        $spoof\_Stat\_id \leftarrow 1$ 
9:     end if
10:  end procedure
11:  procedure CHECKIP()
12:     $R\_IP \leftarrow \text{ExtractIP}(IPMACTable)$ ;
13:    if  $R\_IP == IP.vif\_id$  then ▷ check for real IP in packet
14:       $flag\_id \leftarrow 0$ 
15:    else
16:       $flag\_id \leftarrow 1$ 
17:       $spoof\_Stat\_id \leftarrow 1$ 
18:    end if
19:  end procedure
20:   $DomName\_value \leftarrow \text{getDomName}()$ ;
21:   $DomID\_value \leftarrow \text{getDomId}()$ ;
22:   $vif\_id \leftarrow DomID\_value$ ;
23:  for each  $mac$  and  $IP \in vif(packet)$  do
24:     $CheckMac(mac)$ ;
25:     $CheckIP(IP)$ ;
26:    if  $mac.spoof\_Stat[id] = 1$  then
27:       $BlockVM(id)$ ;
28:       $Print "Spoof Packets Detected DomainName : \$DomName Domainid :$ 
29:       $\$DomID with mac address as spoofed mac = \$mac$ 
30:    end if
31:    if  $IP.spoof\_Stat[id] = 1$  then ▷ Also this status is passed to ARM
32:       $BlockVM(id)$ ;
33:       $Print "Spoof Packets Detected Domain Name : DomName Domain id :$ 
34:       $DomID IP address : spoofed IP = IP$ 
35:    end if
36:    if  $flag\_id == 0$  then
37:       $flag\_id \leftarrow NTBA(packet)$ ;
38:    end if
39:  end for
40: end procedure

```

3.3 Network traffic behavior analyzer (NTBA)

NTBA is responsible for analyzing network activities and detecting malicious network attack patterns. It performs in two stages: (i) Log Pre-processing (ii) Behavior analysis of traffic. Let us explain the working mechanism of each of the stages below:

3.3.1 Log pre-processing. The captured traffic logs by VNTC are in an unstructured format and represent basic network features. Hence, log pre-processing is invoked to extract various other important derived network features and convert the unstructured logs

into structured logs. It uses the open-source network feature extraction tool `pcap_extract_features`⁷ to extract 45 network features. All the captured packets are converted into feature vectors and stored in a feature vector matrix (.csv). The dataset may still contain noise, missing values, or other unwanted information. Hence, it is further processed for handling such issues. The final dataset contains records with one of the following class labels: (i) benign and (ii) attack.

3.3.2 Behaviour identification of network traffic. A deep neural network (DNN) is selected for the classification purpose to detect malicious activity on the basis of performance (discussed in Section 4.3). DNN is a supervised learning which consists of multiple layers of neurons. These neurons extract information from the input and then pass it to the next layer. The layers are arranged so that the first layer (called the input layer) receives the input, which is then processed and passed to the hidden layers and eventually to the output layer. The advantage of using DNN is that it captures the complex relationships in the data, which conventional machine learning models fail to capture. The DNN model is trained with the extracted features so that it can classify malicious packets. The trained DNN model contains seven layers: one input layer, five hidden layers, and one output layer. The number of neurons in the first hidden layer is 512, which is decreased by half, i.e., 256 in the second layer. The third, fourth, and fifth hidden layers contain 128, 64, and 32 neurons, respectively. The activation function used in each hidden layer is *Relu* function, whereas the softmax activation function is used in the output layer. The mathematical formula of *Relu* is given by

$$f(x) = \max(0, x) \quad (1)$$

Similarly, the mathematical formula of *softmax* is given by

$$\sigma(z) = 1/(1 + e^{-z}) \quad (2)$$

Adam optimizer is used as an optimizer for the DNN model. If any packet from the victim VM is identified as "Malicious" by the NTBA module, the status of the packet is stored in the database as "Tested", and the virtual machine is marked with a tag, i.e., "Compromised VM". The status of packets is then passed to the ARM module.

⁷https://github.com/fx2301/packet_capture_feature_extraction

3.4 Alert, report, and mitigation (ARM)

The ARM module is the after-most module. Its main objective is to alert the administrator and mitigation of the compromised VM. The ARM module receives the status and specifics of all the virtual machines. The log includes the VM's domain ID, domain name, VNI information, and specific information about the malicious packet coming from the VM. The port information is taken from the malicious packet headers after the specifics of the machine producing the malicious traffic. The ports' connected services are located and labeled as suspicious. The administrator gets a report of suspicious activity for the particular virtual machine containing the specifics of the compromised VM. On the other hand, a warning is also given to the domain user to take the apps down, and the current traffic flow is obstructed. If a tenant VM produces a lot of malicious traffic, it is isolated or reverted to a checkpoint where it was still benign.

To understand the working of the ARM module, let us discuss a case of spoofed address generation and then the mitigation performed by the ARM module. A scenario can be considered in which a compromised VM with registered IP as "192.168.23.2" broadcasts network requests to all the VMs at the same time with spoofed IP address as "192.168.31.1" using testing tools such as hping3⁸. In such case, there is a chance of conflict among VMs to respond to the received request and handle it synchronously. The proposed system VNSecure oversees such cases by providing a validation layer for the MAC and IP of all the available VM. If Any unrecognized address that is unknown for IP MACtable is found, it is marked as spoofed by the VNTV module, and its detail is forwarded to the ARM module. The ARM module blocks the network flow from the particular domain using the IP tables command to block network traffic from the VM. A rule can be added to the INPUT chain of the filter table to drop all incoming packets from a specific IP address or subnet associated with the VM. A rule for IP tables can be defined with the command:

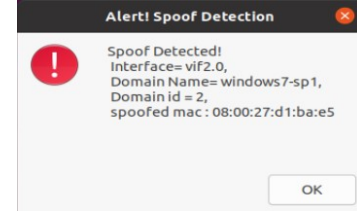
```
IPtables - A INPUT -s < source - IP > -- dport
< port - number > -j DROP
```

The ARM module can distinguish the different spoofing conditions and produce alerts accordingly. The figure 2a shows an alert message in the condition of spoofed mac address and similarly figure 2b shows an alert message in case of detection of spoofed ip address.

4 EXPERIMENTS AND RESULTS

This section discusses the experiment setup, and system configuration, along with the dataset that has been used in the development of the proposed framework. This section also explains the results and model evaluation with the help of concepts of explainable AI. There can be many factors on which the classification of the network traffic may depend. Understanding these features' role in classification plays an important role in developing a trustworthy model.

⁸<https://www.kali.org/tools/hping3/#/protect/protect/leavevmode@ifvmode/kern+.2222em/relax-~|protect/protect/leavevmode@ifvmode/kern+.2222em/relaxtext=hping3%20is%20a%20network%20tool,transfer%20files%20under%20supported%20protocols>



(a) ARM output for spoofed mac



(b) ARM output for spoofed IP detection

Figure 2: ARM output for traffic validation

4.1 Testbed set up details

A testbed environment has been created in the laboratory to validate the proposed detection model using malware and benign binaries. We have configured five bare metal machines having Xen VMM, core i7 processor, 8 GB RAM, and 500 GB SSD Ubuntu 20.04.05 LTS in the lab. Two Windows VMs have been created on top of Xen in each machine to set up a virtual environment. Open-source introspection libraries such as LibVMI [16], Volatility[16], and Drakvuf have been deployed at the hypervisor to inject and execute the malware executable in the victim VM. The network introspection has been performed at Dom0 using tcpdump tool, which sniffs the traffic flowing through the virtual bridge by listening. A high-end server having 32 GB RAM, 500 GB SSD, an Intel Xenon processor, and a guest OS (ubuntu 22.04.01 LTS) is used to analyze the logs. The jupyter notebook and google collab platform is used for ML implementation.

4.2 Dataset

Two datasets have been used in our research; one is the most recent Amazon Web Service(AWS) based network flow dataset, CIC-IDS2017[15]. It was developed through a joint effort between the Communications Security Establishment (CSE) and the Canadian Institute for Cybersecurity (CIC). The dataset comprises seven attack scenarios: Brute-force, Heartbleed, Botnet, DoS, DDoS, Web attacks, and network infiltration from inside. The second dataset is the self-generated dataset, MALNetwork2023, developed in the test bed of our laboratory using malware executables. The malware executable has been taken from online sources. The malware executables belong to *zbot*, *vundo*, *vobfus*, and *zeroaccess* families. Our approach considers all these attack scenarios malicious traffic, while safe traffic is deemed benign. The dataset comprises a total of 80 features. These features include timestamps which indicate the time when the packet was captured; payload, which contains

Table 1: Discussion on results achieved using different classifiers and comparative analysis with other-state-of-art

Approach	Dataset	Accuracy	TPR	TNR	FPR	FNR
Logistic Regression	CICIDS2017	90.43%	70.50%	97.73%	2.26%	29.49%
K-Nearest Neighbor	CICIDS2017	97.46%	86.70%	92%	1.7%	7.9%
Decision Tree Classifier	CICIDS2017	98.39%	89.22%	99.17%	0.82%	10.77%
Random Forest	CICIDS2017	98.48%	89.22%	99.17%	0.2%	10.77%
Proposed VNSecure	CICIDS2017	99.01%	90.57%	97.35%	0.45%	8.9%
LR	MALNetwork2023	76.84%	97%	89%	2.5%	11%
RF	MALNetwork2023	86.59%	85%	88%	15%	12%
DT	MALNetwork2023	89%	94%	84%	6%	16%
KNN	MALNetwork2023	89.44%	87%	92%	13%	7.9%
Proposed VNSecure	MALNetwork2023	93.2%	86.03%	81.01%	5.7%	23.7%
Abbar et. al [2]	NSL-KDD	99.47	-	-	-	-
Abd et al. [1]	UNSW-NB15	92.5	-	-	-	-
Mishra et al. [12]	UNSW-NB15	95.091	-	-	2.415	-

the actual data being transmitted, can be extracted, etc. The dataset was processed and prepared using VNSecure modules.

4.3 Results and Discussion

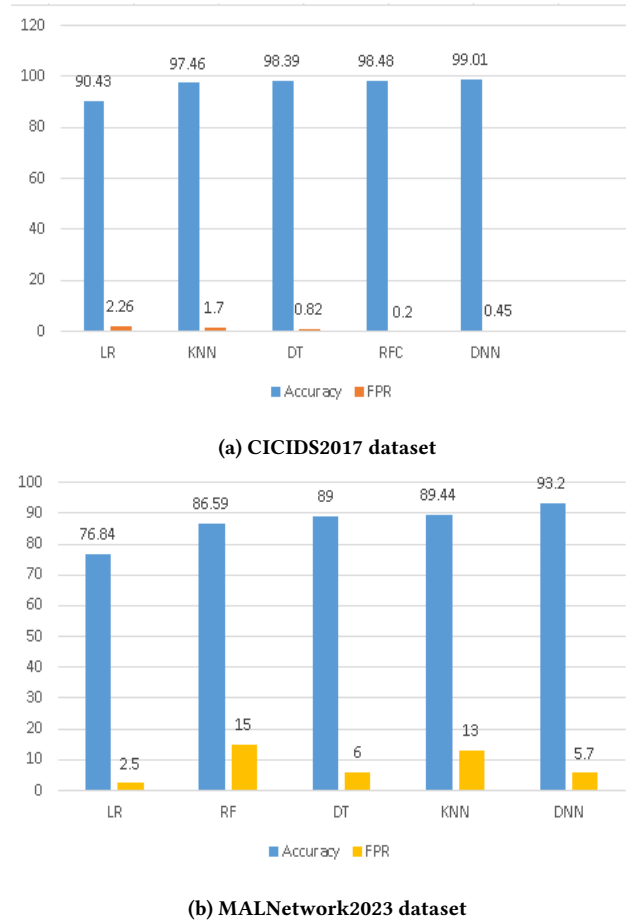


Figure 3: The analysis of the result obtained through two different datasets

In this section, VNSecure is tested and validated using publicly available and self-generated datasets. Results are evaluated based on different parameters, including accuracy, TPR, TNR, FPR, and

FNR. Five traditional machine learning models, including k-nearest neighbor (KNN), decision tree (DT), random forest (RF), logistic regression (LR), and one deep learning-based algorithm (Deep neural network (DNN)) have been used to analyze the virtual traffic behavior. Let us first discuss the result obtained using the CICIDS2017 dataset (publicly available) in conventional and deep learning classifiers. LR was initially applied, which provided 90.43% accuracy with 70.50% TPR, 97.73% TNR, 2.26% FPR, and 29.49% FNR. LR captures only linear relationships among features; complex patterns among features should be captured to improve performance. A non-parametric classifier, i.e., KNN, is selected to enhance the performance. KNN improves the accuracy by 7.03% and TPR by 16.20%, as shown in Figure 3a. Since KNN fails to handle non-linear datasets, two tree-based algorithms improve the result. The decision tree shows an improvement of 0.93% in classification accuracy compared to KNN. The decision tree may face the issue of overfitting, so the random forest classifier is then selected to improve the result. In the random forest, a rise of 0.09% is observed in the accuracy compared to DT. The random forest may also face overfitting if either number of trees is large or the depth of the tree is deep. So, a DNN model has been opted for, which improves classification accuracy by 0.53% as compared to RF, as shown in Table 1.

Let us discuss the result of VNSecure obtained using the MALNetwork2023 dataset, which is generated by validating all the modules of the proposed framework. Initially, LR was used, which provided 76.84% accuracy with 97% TPR. To improve the result, RF was selected, which raised the accuracy by 9.75% as compared to LR. It is observed that DT and KNN both performed better than RF by 2.41% and 2.885%, respectively. To improve the result further, DNN is selected for the apparent reason. In DNN, an accuracy of 93.2% is achieved, which shows an improvement of 3.76% when compared to KNN as shown in Figure 3b.

The proposed VNSecure is compared with other states of art approaches, such as Abbar et al. [2], Abd et al. [1] and Mishra et al. [12]. Abbar et al. proposed an intrusion detection model which is evaluated using NSL-KDD⁹ and provides 99.47% accuracy. The approach is applicable in the traditional network environment. Abd et al. proposed an intrusion detection model that provides an accuracy of 92.5% in the UNSW-NB15¹⁰ dataset. The proposed VNSecure achieved 99.01% accuracy in the CICIDS2017 dataset while 93.2% in MALNetwork2023. Mishra et al. introduced an approach

⁹<https://www.unb.ca/cic/datasets/ns1.html>

¹⁰<https://research.unsw.edu.au/projects/uns-w-nb15-dataset>

0.1214 ± 0.0016	Flow IAT Min
0.0886 ± 0.0013	Bwd Pkts/s
0.0603 ± 0.0012	Init Bwd Win Byts
0.0597 ± 0.0005	Fwd IAT Min
0.0540 ± 0.0010	Init Fwd Win Byts
0.0525 ± 0.0008	Pkt Len Mean
0.0484 ± 0.0008	Bwd Pkt Len Max
0.0481 ± 0.0007	Flow IAT Max

Figure 4: Explainability of *proposed VNSecure* using ELI5 library

that got 95.091% accuracy in the UNSW-NB15 dataset. One of the significant limitations of the abovementioned approaches is that older datasets have been used to train the IDS model. The older datasets do not reflect the evolving attack behavior of an adversary. The proposed *VNSecure* is trained on a relatively new dataset which may help to detect sophisticated attacks.

It is observed that the above mentioned approaches have been validated on the dataset generated in the physical network environment. It should be noted that the dataset generated in physical networks and virtual networks may differ. Another key feature of the proposed framework is that *VNSecure* is validated on the virtualization-enabled dataset. Moreover, the other above mentioned IDS approaches work as a black box model. The proposed *VNSecure* provides an insight into the working of the DNN model by utilizing XAI libraries such as ELI5 and SHAP.

It is crucial to know which features are influential in deciding the outcome made by the ML/DL model. Since the proposed model uses DNN, which is a well-known black box model, it becomes hard to understand the working of the model. To tackle the abovementioned issue, the results obtained through *VNSecure* is further analyzed using XAI libraries, more specifically ELI5 (Explain me like 5) and Shapley Additive exPlanations (SHAP).

ELI5 is a Python library that uses permutation importance to compute the contribution of each feature in the prediction of a particular class label. The idea behind using permutation importance in ELI5 is to find the relative importance among features. Accuracy is calculated before and after shuffling features. If there is an increment in the accuracy after the shuffle, the feature is essential in the decision-making of the DNN model. For example, It can be observed from Figure 4, *Forward Segment packet* has the highest contribution (0.2775) in the prediction of labels. In contrast, the feature indicating the minimum length of the backward packet has the least importance in the prediction made by the DNN model.

SHAP computes the shapely values, which is the mean of the marginal contribution of each feature. For the global interpretation of *VNSecure*, a shap summary plot is used to visualize the prediction made by the DNN model. A shap summary plot is used to display the mean SHAP values for each feature in their corresponding classes in the form of a bar graph. The shap values are calculated using the formula

$$\phi_i = (1/N) \left[\sum_{j=1}^N [f(x_j) - E(f(x))] * \delta_i(x_j) \right] \quad (4)$$

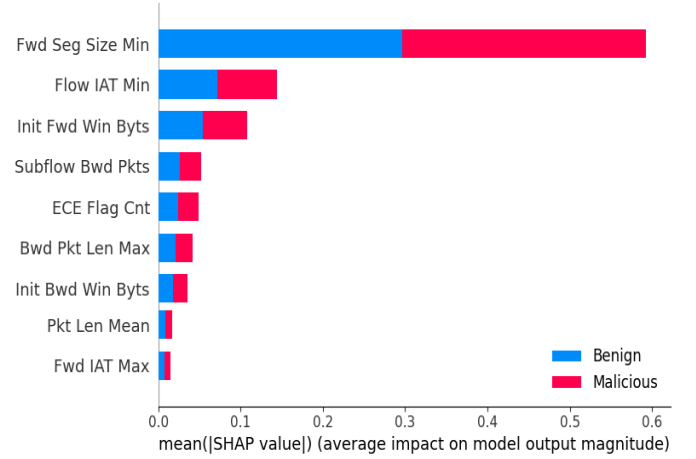


Figure 5: Explainability of *proposed VNSecure* using SHAP library

where ϕ_i refers to the shap value of i^{th} feature, N is the total number of features, (x_j) is the j^{th} instance in the dataset, f is the classifier, $E(f(x))$ is the expectation of the f and $\delta_i(x_j)$ is the difference in the i_{th} and j_{th} instance of the dataset.

The high value of the mean SHAP values indicates the higher contribution of the feature. For example, *Fwd Seg Size Min* has the highest mean SHAP values (0.6), which indicates the highest contribution for both classes (i.e., benign and malicious) in the prediction made by the DNN model as shown in Figure 5. It can be observed that *Fwd IAT Max* and *Pkt len Mean* have the lowest shap values and hence, minimal influence on the decision made by the DNN model.

5 CONCLUSION

We proposed a security approach, *VNSecure*, to prevent and identify malicious network packets. The approach is implemented at Dom0 in the Virtualization layer and involves capturing virtual network traffic using Virtual Network Traffic Capturing (VNTC). The first level of security, Virtual Network Traffic Validator (VNTV), identifies the IP and MAC addresses of the network packets, while the second level of security, Virtual Traffic Behaviour Analyser (VTBA), analyzes the packets' behavior. The VTBA module forwards the status of the packets to the ARM module, which blocks network traffic from suspicious sources and maintains the IP and MAC addresses of compromised domains. Only verified packets are forwarded to the destination. The approach employs a machine learning algorithm for behavior analysis to detect anomalies in the network traffic. By incorporating these two layers of security checks, the precision of the approach is heightened, rendering it appropriate for real-time scenarios.

ACKNOWLEDGMENTS

The Science and Engineering Research Board, Department of Science and Technology (SERB-DST) provided funding for this study under Project File No. SPG/2021/002003. The authors would like to

express their gratitude to SERB-DST for their intellectual generosity and research assistance.

REFERENCES

- [1] Shamis N Abd, Mohammad Alsajri, and Hind Raad Ibraheem. 2020. Rao-SVM machine learning algorithm for intrusion detection system. *Iraqi Journal For Computer Science and Mathematics* 1, 1 (2020), 23–27.
- [2] Iram Abrar, Zahrah Ayub, Faheem Masoodi, and Alwi M Bamhdi. 2020. A machine learning approach for intrusion detection system on NSL-KDD dataset. In *2020 international conference on smart electronics and communication (ICOSEC)*. IEEE, 919–924.
- [3] David Chisnall. 2007. *The Definitive Guide to the Xen Hypervisor* (first ed.). Prentice Hall Press, USA.
- [4] Muhammad Shahid Dildar, Nayeem Khan, Johari Bin Abdullah, and Adnan Shahid Khan. 2017. Effective way to defend the hypervisor attacks in cloud computing. In *2017 2nd International Conference on Anti-Cyber Crimes (ICACC)*. 154–159. <https://doi.org/10.1109/Anti-Cybercrime.2017.7905282>
- [5] Jonghoo Han and Wooguil Pak. 2023. High Performance Network Intrusion Detection System Using Two-Stage LSTM and Incremental Created Hybrid Features. *Electronics* 12, 4 (2023), 956.
- [6] Hårek Haugerud, Huy Nhut Tran, Nadjib Aitsaadi, and Anis Yazidi. 2021. A dynamic and scalable parallel Network Intrusion Detection System using intelligent rule ordering and Network Function Virtualization. *Future Generation Computer Systems* 124 (2021), 254–267.
- [7] Taehoon Kim and Wooguil Pak. 2022. Robust network intrusion detection system based on machine-learning with early classification. *IEEE Access* 10 (2022), 10754–10767.
- [8] Tamas K Lengyel, Steve Maresca, Bryan D Payne, George D Webster, Sebastian Vogl, and Aggelos Kiayias. 2014. Scalability, fidelity and stealth in the DRAKVUF dynamic malware analysis system. In *Proceedings of the 30th annual computer security applications conference*. 386–395.
- [9] Scott M Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. *Advances in neural information processing systems* 30 (2017), 1–10.
- [10] Kuriakose Mathew, Purushottam Kulkarni, and Varsha Apte. 2010. Network bandwidth configuration tool for Xen virtual machines. In *2010 Second International Conference on COMMunication Systems and Networks (COMSNETS 2010)*. IEEE, 1–2.
- [11] Preeti Mishra, Emmanuel S Pilli, Vijay Varadharajan, and Udaya Tupakula. 2017. Intrusion detection techniques in cloud environment: A survey. *Journal of Network and Computer Applications* 77 (2017), 18–47.
- [12] Preeti Mishra, Emmanuel S Pilli, Vijay Varadharajan, and Udaya Tupakula. 2017. Out-VM monitoring for malicious network packet detection in cloud. In *2017 ISEA asia security and privacy (ISEASP)*. IEEE, 1–10.
- [13] Preeti Mishra, Emmanuel S Pilli, Vijay Varadharajan, and Udaya Tupakula. 2017. PSI-NetVisor: Program semantic aware intrusion detection at network and hypervisor layer in cloud. *Journal of Intelligent & Fuzzy Systems* 32, 4 (2017), 2909–2921.
- [14] Chirag Modi and Dhiren Patel. 2018. A feasible approach to intrusion detection in virtual network layer of Cloud computing. *Sādhanā* 43 (2018), 1–16.
- [15] Canadian Institute of Cybersecurity. 2017. IDS 2017 | Datasets | Research | Canadian Institute for Cybersecurity | UNB. <https://www.unb.ca/cic/datasets/ids-2017.html>. (Accessed on 05/02/2023).
- [16] Bryan D Payne. 2012. *Simplifying virtual machine introspection using LibVMI*. Technical Report. Sandia National Laboratories (SNL), Albuquerque, NM, and Livermore, CA
- [17] Pratikkumar Prajapati and Mark Stamp. 2021. An empirical analysis of image-based learning techniques for malware classification. *Malware analysis using artificial intelligence and deep learning* (2021), 411–435.
- [18] Kicho Yu, Khanh Nguyen, and Younghee Park. 2022. Flexible and Robust Real-Time Intrusion Detection Systems to Network Dynamics. *IEEE Access* 10 (2022), 98959–98969.
- [19] Feng Zhao, Weiping Yang, Hai Jin, and Song Wu. 2008. VNIDS: A virtual machine-based network intrusion detection system. In *2008 2nd International Conference on Anti-counterfeiting, Security and Identification*. IEEE, 254–259.